

Manejador de Recursos Distribuidos para HPC.

**Franco Franzini,
Santos David Lurati,
Gianfranco Morán,
Maximiliano Zambón.**

Sistemas Operativos.

**Facultad de Ciencias Exactas
Ingeniería y Agrimensura.**

22/6/2026.

1. Introducción

El proyecto implementa un middleware distribuido que gestiona los recursos (CPU, GPU y memoria) de un clúster HPC simulado. Cada nodo es autónomo y se compone de dos procesos que cooperan a través de una interfaz local.

El agente de recursos en C administra los recursos físicos del nodo, atiende pedidos de otros agentes por TCP y descubre nodos de la red por broadcast UDP; usa epoll con varios hilos para manejar muchas conexiones sin bloquearse. El planificador en Erlang genera la carga de trabajo simulada, decide qué recursos pedir y a qué nodos, y aplica la estrategia contra deadlocks.

El desafío central es la coordinación distribuida: reservar recursos que viven en máquinas distintas sin bloquear el sistema ante un deadlock.

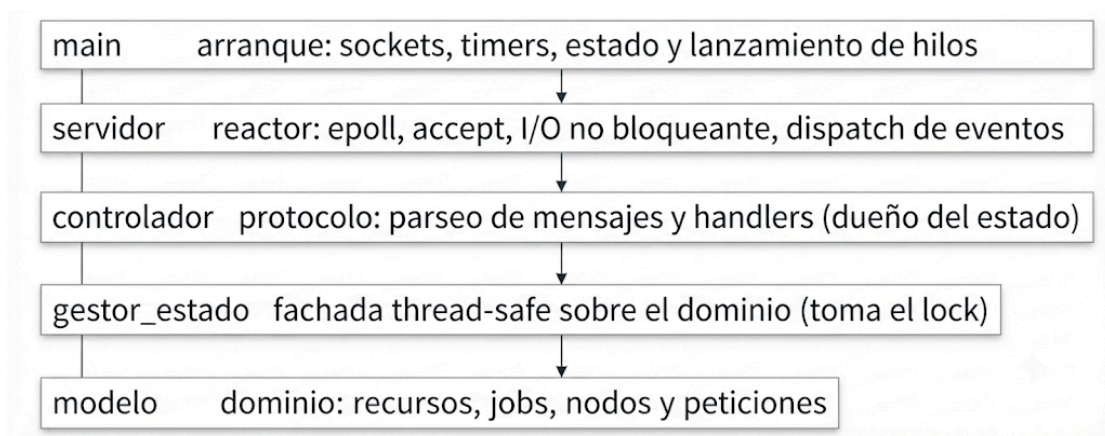
2. Agente de recursos en C

Roles simultáneos

El agente actúa a la vez como servidor de sus recursos locales (responde RESERVE de otros nodos), como cliente cuando su Erlang necesita recursos remotos (abre una conexión TCP y envía RESERVE), y como puente que traduce el protocolo con Erlang (los mensajes JOB) al protocolo entre agentes (RESERVE, GRANTED, DENIED, RELEASE).

Organización en capas

El código está pensado en capas donde cada una conoce lo mínimo de la de abajo. El servidor no sabe de protocolo ni de estado; el controlador no sabe de epoll salvo para pedir "envía esto" o "abrí una conexión".



Concurrencia: epoll con múltiples hilos

Al arrancar, el main crea el epoll, los sockets de escucha (TCP local para Erlang, TCP público para otros agentes, UDP para descubrimiento) y dos timers. Luego lanza 4 hilos que corren el mismo bucle de eventos. Para que el modelo multihilo sea correcto se usan dos banderas del epoll. Los sockets de escucha y los timers, que son descriptores compartidos y permanentes, se registran con EPOLLEXCLUSIVE, de modo que el kernel despierta a un solo hilo por evento. Los clientes conectados se registran con EPOLLONESHOT, que garantiza que a lo sumo un hilo procese a un cliente a la vez; cuando termina de leerlo, el cliente se re-arma en el epoll.

La escritura es no bloqueante y es la contraparte de EPOLLOUT: si al enviar un mensaje el kernel no lo acepta entero, el remanente se guarda en un buffer de salida del cliente y se arma EPOLLOUT; cuando el socket vuelve a tener lugar, el bucle drena ese buffer. Así ningún hilo se bloquea escribiendo y no se pierden mensajes. Cada cliente tiene su propio lock de salida porque cualquier hilo puede escribirle.

Gestión de recursos locales

Cada recurso local (CPU, GPU, memoria) tiene una capacidad total, una cantidad disponible y una cola FIFO de solicitudes pendientes. Ante un RESERVE, si hay disponible y la cola está vacía se descuenta y se responde GRANTED; si no, la solicitud se encola. Ante un RELEASE se devuelve la cantidad y se atiende la cola en orden, concediendo a los siguientes mientras alcancen los recursos.

El agente lleva un libro contable de los jobs activos con los recursos que cada uno tiene concedidos. Esto permite liberar todo lo de un job ante una desconexión abrupta de su socket, devolviendo esos recursos y repartiéndolos a la cola. Tanto el registro de nodos como el de jobs se guardan a la vez en una tabla hash (búsqueda eficiente) y en una lista enlazada (recorrido), combinando lo mejor de ambas.

Descubrimiento de nodos por UDP

Cada agente emite un ANNOUNCE por broadcast cada 5 segundos con sus recursos disponibles; la IP se extrae del propio datagrama con recvfrom. Cada nodo mantiene una tabla con IP, puerto, recursos y timestamp del último anuncio, y un nodo que no se anuncia durante 15 segundos se elimina. Al iniciar, el agente emite su anuncio y espera 2 segundos para poblar la tabla con los nodos ya activos antes de atender pedidos.

3. Planificador en Erlang

El planificador aprovecha el modelo de actores: procesos livianos y aislados que se comunican por mensajes.

Cada job es un proceso independiente. El receive bloqueante que espera la respuesta (granted, denied o timeout) vive en el worker, así el scheduler nunca se frena atendiendo a otras fuentes como las respuestas del agente, los nuevos jobs o la actualización de nodos. Si un worker crashea, el scheduler lo detecta, limpia su mapa de jobs y sigue vivo.

Las responsabilidades están separadas: los workers piden cantidades abstractas de cada recurso, y el scheduler, que es el que mantiene la lista de nodos actualizada, traduce eso a asignaciones concretas (a qué IP, qué recurso y cuánto) antes de mandarlas al agente. Para esa traducción usa un algoritmo voraz: si un job pide más de lo que un solo nodo tiene, reparte la demanda entre varios nodos, recorridos en orden aleatorio, hasta cubrirla; si no alcanza, deniega el job localmente sin siquiera molestar al agente.

Alrededor del scheduler corren dos servicios de soporte, un generador de trabajos y un actualizador de nodos que pide la lista al agente cada 15 segundos. Ambos están enlazados al scheduler, de modo que si este muere ellos también. Un tope de jobs simultáneos evita saturar clústeres chicos. Ante una desconexión, el proceso principal reintenta la conexión con backoff exponencial y un techo de 30 segundos. El logging lo maneja un único proceso registrado, que serializa la escritura de concesiones, denegaciones y timeouts. Por último, la conexión con el agente usa modo de framing por líneas, de modo que cada lectura devuelve exactamente un mensaje y el parser puede asumir un comando por vez.

4. Protocolos

Todos los mensajes son líneas de texto ASCII terminadas en salto de línea.

Desde Erlang hacia el agente hay tres mensajes: GET_NODES pide la tabla de nodos; JOB_REQUEST seguido del id del job y de una lista de tokens del tipo @IP:recurso:cantidad solicita reservas en uno o varios nodos; y JOB_RELEASE

seguido del id libera todo lo que ese job tenía reservado. En sentido inverso, el agente responde `JOB_GRANTED` cuando todos los nodos concedieron, `JOB_DENIED` cuando un nodo rechazó un pedido imposible (caso en que Erlang no reintenta) y `JOB_TIMEOUT` cuando la petición no se completó en 30 segundos (caso en que Erlang sí reintenta).

Entre agentes de C el protocolo tiene cuatro mensajes: `RESERVE` con el id, el recurso y la cantidad; `GRANTED` y `DENIED` como respuestas a ese pedido; y `RELEASE` para cancelar o liberar una reserva.

La respuesta a `GET_NODES` tiene la forma `NODES IP:puerto:cpu:X:mem:Y:gpu:Z`, separando varios nodos con punto y coma. El descubrimiento por UDP usa el mensaje `ANNOUNCE` con el puerto y los recursos disponibles del nodo.

5. Estrategia contra deadlock

Elegimos detección por timeout con reintento y espera aleatoria, y no prevención por ordenamiento de recursos, porque esta última exigiría un orden canónico de recursos acordado entre todos los equipos, incluidos los de otros grupos, que no podemos garantizar.

La detección vive en el agente C y opera en dos planos. El timer de mantenimiento dispara cada 15 segundos y aplica el timeout de 30 segundos en dos lugares. Del lado dueño del recurso, un `RESERVE` ajeno que quedó encolado más de 30 segundos se desencola en silencio; es higiene de cola y no notifica a nadie. Del lado coordinador, si una petición que este agente coordina no se completó a tiempo, el agente hace rollback enviando `RELEASE` a todos sus nodos y avisa a su Erlang con `JOB_TIMEOUT`.

Un punto clave del diseño es que no hay ningún mensaje de timeout entre agentes: el coordinador lo detecta por sí mismo mirando el timestamp de su petición. Así el protocolo entre agentes queda limpio y el `DENIED` se reserva para el rechazo genuino de un pedido imposible (recurso desconocido o cantidad mayor a la capacidad total).

Del lado de Erlang, el contrato distingue los dos casos. Un `JOB_DENIED` es un fallo permanente y el worker no reintenta, porque un pedido imposible no mejora reintentando. Un `JOB_TIMEOUT` es congestión o deadlock, así que el

worker espera un tiempo aleatorio y reintenta con nuevas cantidades. La espera aleatoria es el mecanismo central: rompe la simetría entre dos jobs que compiten por los mismos recursos, haciendo improbable que ambos reintentos vuelvan a chocar.

Ejemplo con dos nodos. El Nodo A ofrece cpu:2 y el Nodo B ofrece gpu:1. El Job 1001 quiere cpu de A y gpu de B; el Job 2001 quiere lo mismo en orden cruzado.

Recursos: Nodo A (cpu:2), Nodo B (gpu:1)

Peticiones:

Job 1001: RESERVE cpu -> GRANTED y RESERVE gpu -> ENCOLADO

Job 2001: RESERVE cpu -> ENCOLADO y RESERVE gpu -> GRANTED

Deadlock:

Job 1001 tiene cpu@A y espera gpu@B

Job 2001 tiene gpu@B y espera cpu@A -> Deadlock

Resolución por timeout:

A los 30 s cada coordinador expira su petición: RELEASE a sus nodos + JOBTIMEOUT a su Erlang (en paralelo, cada nodo desencola el RESERVE ajeno vencido)

Reintento: Erlang reintenta con una espera aleatoria distinta: uno adquiere todo primero y libera el camino para el otro

En cuanto a sus propiedades, la estrategia garantiza liveness porque el deadlock se rompe con el timeout más el reintento aleatorio, con el único riesgo teórico de livelock, mitigado por la aleatoriedad de la espera. Garantiza safety porque no hay doble asignación: el coordinador libera antes de avisar, y un GRANTED que llega tarde se ignora porque la petición ya no está en la tabla. Y todo ocurre sin coordinación global: cada agente y cada worker actúan por su cuenta, sin mensajes de timeout entre agentes ni estado compartido.

6. Roles del equipo

Ingeniero de comunicaciones en C: Maximiliano Zambón.

Gestor de recursos y estado: Santos Lurati.

Planificador y lógica anti-deadlock: Gianfranco Morán / Franco Franzini.

Integración, pruebas e interoperabilidad: Gianfranco Morán / Franco Franzini.